

Algorithms and Data Structures

07.02.2023

Exercise 1 (1.5 points)

Given the following array of integer values, perform the first step of quicksort to sort the array in ascending order; then, from the initial array, generate the right and the left partitions.

10 9 11 4 1 4 2 3 7 5

Report 3 integer values: The pivot selected on the original array, the pivot you would select on the left partition generated from the original array, and the pivot you would select on the right partition generated (again) from the original array. No other symbols must be included in the response. This is an example of the response format: 13 1 10

Solution

```
### Quick sort
Solution format 1:
Initial array: 10 9 11 4 1 4 2 3 7 5
Re l= 0;r= 4: 3 2 4 4 1
Re l= 0;r=-1:
Re l= 0;r=-1: 2 4 4 3
Re l= 1;r= 1: 2
Re l= 1;r= 1: 4 4
Re l= 3;r= 2:
Re l= 3;r= 2: 4
Re l= 0;r= 4: 9 10 7 11
Re l= 6;r= 8: 9 10 7
Re l= 6;r= 5:
Re l= 6;r= 5: 10 9
Re l= 7;r= 6:
Re l= 7;r= 6: 10
Re l= 6;r= 8:
Solution format 2:
Input array: [0]10 [1]9 [2]11 [3]4 [4]1 [5]4 [6]2 [7]3 [8]7 [9]5
Pivot: [9]5
Swaps: [0<->7]10<->3 - [1<->6]9<->2 - [2<->5]11<->4 - [5<->9]11<->5
Output array: [0]3 [1]2 [2]4 [3]4 [4]1 - [5]5 - [6]9 [7]10 [8]7 [9]11
Input array: [0]3 [1]2 [2]4 [3]4 [4]1 - [6]9 [7]10 [8]7 [9]11
Pivot: [4]1 - [9]11
Swaps: [0<->4]3<->1 - [9<->9]11<->11
Output array: - [0]1 - [1]2 [2]4 [3]4 [4]3 - [6]9 [7]10 [8]7 - [9]11 -
Input array: [1]2 [2]4 [3]4 [4]3 - [6]9 [7]10 [8]7
Pivot: [4]3 - [8]7
Swaps: [2<->4]4<->3 - [6<->8]9<->7
Output array: [1]2 - [2]3 - [3]4 [4]4 - - [6]7 - [7]10 [8]9
Input array: [3]4 [4]4 - [7]10 [8]9
Pivot: [4]4 - [8]9
Swaps: [3<->4]4<->4 - [7<->8]10<->9
Output array: - [3]4 - [4]4 - - [7]9 - [8]10
```

Response

5 1 11

Exercise 2 (1.5 points)

Consider a binary tree whose visits return the following sequences.

```
Pre-order: A B D F H G E C I L
In-order: H F D G B E A I L C
Post-order: H F G D E B L I C A
```

Report the sequence of keys stored on the tree leaves, moving on the tree from left to right. Please, report the list of keys on the same line, separated by a single space. No other symbols must be included in the response. This is an

example of the response format: A X C Y etc.

Solution

```
### Tree guess
Node Key Left Right
1      A      2      3
2      B      4      5
3      C      6 NULL
4      D      7 NULL
5      E NULL NULL
6      I NULL 9
7      F 10 NULL
8      G NULL NULL
9      L NULL NULL
10     H NULL NULL
```

Response

H G E L

Exercise 3 (1.5 points)

Insert the following sequence of keys into an initially empty hash table. The hash table has a size equal to $M=19$. Insertions occur character by character using open addressing with quadratic probing (with $c_1 = 1$ and $c_2 = 1$). Each character is identified by its index in the English alphabet (i.e., $A=1, \dots, Z=26$). Equal letters are identified by a different subscript (i.e., A and A become A1 and A2).

R I C C I A

Indicate in which elements are placed the last three letters of the sequence, i.e., C, I, and A, in this order. Please, report your response as a sequence of integer values separated by one single space. No other symbols must be included in the response. This is an example of the response format: 3 4 11

Solution

```
### hash-table
Character keys
Hash table size = 19
Number of keys = 6
Quadratic probing: c1=1.000000, c2=1.000000.
Key: R I C C I A
Key=R k=018 Final=18
Key=I k=009 Final=09
Key=C k=003 Final=03
Key=C k=003 Coll.= 3 Final=05
Key=I k=009 Coll.= 9 Final=11
Key=A k=001 Final=01
00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18
- A - C - C - - - I - I - - - - - - R
```

Response

5 11 1

Exercise 4 (1.5 points)

The following capital letters are given with their absolute frequency.

A:4 B:7 C:8 D:10 E:17 F:5

Find an optimal Huffman code for all symbols in the set using a greedy algorithm. Indicate which one of the following encoding is correct.

1. A(01) B(10) C(110) D(1111) E(1110) F(00001)
2. A(010) B(10) C(110) D(1111) E(1110) F(00)
3. A(01) B(100) C(110) D(1111) E(1110) F(00)
4. A(000) B(100) C(101) D(01) E(11) F(001)
5. A(11) B(10) C(110) D(111) E(1110) F(00)

6. A(101) B(110) C(11) D(1111) E(1110) F(00)

Solution

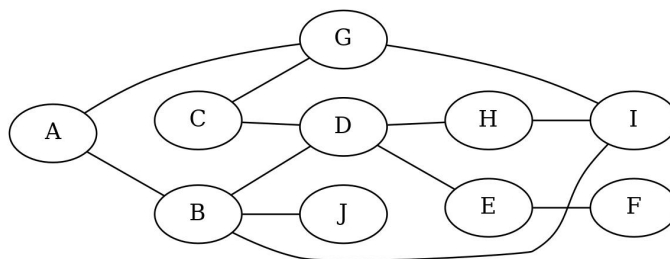
```
### Huffman
Huffman:
Number of codes: 6
A:4 B:7 C:8 D:10 E:17 F:5
Merging: {A} (4) [-1] [-1] + {F} (5) [-1] [-1] = {AF} (9) [0] [1]
Merging: {B} (7) [-1] [-1] + {C} (8) [-1] [-1] = {BC} (15) [2] [3]
Merging: {AF} (9) [0] [1] + {D} (10) [-1] [-1] = {AFD} (19) [4] [5]
Merging: {BC} (15) [2] [3] + {E} (17) [-1] [-1] = {BCE} (32) [6] [7]
Merging: {AFD} (19) [4] [5] + {BCE} (32) [6] [7] = {AFDBCE} (51) [8] [9]
A 000
F 001
D 01
B 100
C 101
E 11
```

Response

4

Exercise 5 (1.5 points)

Given the following graph find all articulation points. If necessary, consider nodes and edges in alphabetical order.



Display all articulation points alphabetically. Please, report only the list of vertices separated by a single space. No other symbols must be included in the response. This is an example of the response format: A B C etc.

Solution

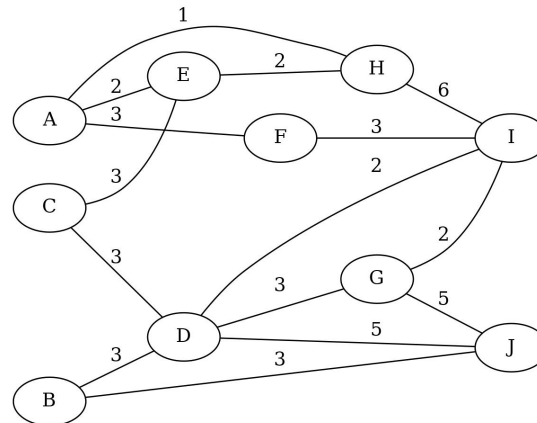
```
UN-directed matrix ... stated as such.
UN-weighted matrix ...stated as such.
Vertex list: [ 0] A [ 1] B [ 2] C [ 3] D [ 4] E [ 5] F [ 6] G [ 7] H [ 8] I [ 9] J
Adjacency Matrix:
0 1 0 0 0 0 1 0 0 0
1 0 0 1 0 0 0 0 1 1
0 0 0 1 0 0 1 0 0 0
0 1 1 0 1 0 0 1 0 0
0 0 0 1 0 1 0 0 0 0
0 0 0 0 1 0 0 0 0 0
1 0 1 0 0 0 0 0 1 0
0 0 0 1 0 0 0 0 1 0
0 1 0 0 0 0 1 1 0 0
0 1 0 0 0 0 0 0 0 0
### Graph Articulation Points and Bridges
Bridge: Edge [ 4] E - [ 5] F
Bridge: Edge [ 3] D - [ 4] E
Bridge: Edge [ 1] B - [ 9] J
Articulation point: Vertex [ 1] B
Articulation point: Vertex [ 3] D
Articulation point: Vertex [ 4] E
```

Response

B D E

Exercise 6 (1.5 points)

Given the following undirected and weighted graph find a minimum spanning tree using Kruskal algorithm.



Indicate the total weight of the final minimum spanning tree. Report one single integer value. No other symbols must be included in the response. This is an example of the response format: 13

Solution

UN-directed matrix ... stated as such.

Weighted matrix ... stated as such.

Vertex list: [0] A [1] B [2] C [3] D [4] E [5] F [6] G [7] H [8] I [9] J

Adjacency Matrix:

```
0 0 0 0 2 3 0 1 0 0
0 0 0 3 0 0 0 0 0 3
0 0 0 3 3 0 0 0 0 0
0 3 3 0 0 0 0 3 0 2 5
2 0 3 0 0 0 0 0 2 0 0
3 0 0 0 0 0 0 0 0 3 0
0 0 0 3 0 0 0 0 0 2 5
1 0 0 0 2 0 0 0 0 6 0
0 0 0 2 0 3 2 6 0 0 0
0 3 0 5 0 0 5 0 0 0 0
```

Graph MST (Kruskal and Prim)

Kruskal

Sorted array of edges: src dst weight:

```
A H 1
A E 2
D I 2
E H 2
G I 2
A F 3
B D 3
B J 3
C D 3
C E 3
D G 3
F I 3
D J 5
G J 5
H I 6
```

MST (list of edges):

```
Edge [ 0] A - [ 0] H ==> weight = 1
Edge [ 0] A - [ 0] E ==> weight = 2
Edge [ 3] D - [ 3] I ==> weight = 2
Edge [ 6] G - [ 6] I ==> weight = 2
Edge [ 0] A - [ 0] F ==> weight = 3
Edge [ 1] B - [ 1] D ==> weight = 3
Edge [ 1] B - [ 1] J ==> weight = 3
Edge [ 2] C - [ 2] D ==> weight = 3
Edge [ 2] C - [ 2] E ==> weight = 3
Total tree weight = 22
```

Prim

```

MST (list of edges):
Edge [ 0]  A - [ 7]  H ==> weight = 1
Edge [ 0]  A - [ 4]  E ==> weight = 2
Edge [ 4]  E - [ 2]  C ==> weight = 3
Edge [ 2]  C - [ 3]  D ==> weight = 3
Edge [ 3]  D - [ 8]  I ==> weight = 2
Edge [ 8]  I - [ 6]  G ==> weight = 2
Edge [ 3]  D - [ 1]  B ==> weight = 3
Edge [ 0]  A - [ 5]  F ==> weight = 3
Edge [ 1]  B - [ 9]  J ==> weight = 3
Total tree weight          = 22

```

Response

22

Exercise 7 (4.0 points)

Define what a Minimum Spanning Tree is and specify the differences from a Single Source Shortest Path problem. Define what a cut, a light edge, and a safe edge are. Clarify why we define safe edges and how the algorithms of Prim and Kruskal use this concept. Are they greedy algorithms? Do they find an optimal solution? There is only one or more than one solutions to the MST problem?

Response

Please, see the teacher's overhead for a complete analysis of the topic.

Exercise 8 (2.0 points)

Analyze the following program and indicate the exact output it generates. Please, report the exact program output with no extra symbols.

```

#define R 3
#define C 5
#define D 3
void f (int[][C]);
int main(void) {
    int mat[R][C] = {
        {1,2,3,4,5},
        {1,2,3,4,5},
        {1,2,3,4,5},
    };
    f (mat);
    return (1);
}

void f (int mat[][C]) {
    int i1, i2, j1, j2, s;
    for (i1=0; i1<=R-D; i1++) {
        for (j1=0; j1<=C-D; j1++) {
            s = 0;
            for (i2=0; i2<D; i2++) {
                for (j2=0; j2<D; j2++) {
                    s = s + mat[i1+i2][j1+j2];
                }
            }
            printf ("%d ", s);
        }
    }
    return;
}

```

Response

18 27 36

Exercise 9 (2.0 points)

The following function allocates a three-dimensional matrix (a two-dimensional matrix of strings) and reads the strings from standard input. The parameters `r` and `c` are the numbers of rows and columns of the matrix.

```

char ***read (int *r, int *c) {

```

```

char word[MAX_LEN], ***mx;    // LINE 1
int i, j;
LINE 2
if (mx == NULL) {
    printf("Memory allocation error.\n");
    exit(EXIT_FAILURE);
}
for (i=0; i<r; i++) {
    mx[i] = (char **)malloc(c * sizeof(char *));
    if (mx[i] == NULL) {
        printf("Memory allocation error.\n");
        exit(EXIT_FAILURE);
    }
}
for (i=0; i<r; i++) {
    for (j=0; j<c; j++) {
        printf("String: ");
        fscanf("%s", word);
        LINE 3
        if (mx[i][j] == NULL) {
            printf("Memory allocation error.\n");
            exit(EXIT_FAILURE);
        }
    }
}
return mx;
}

```

Indicates which ones of the following statements are correct. Note that incorrect answers imply a penalty in the final score.

1. In LINE 1 the array word can be defined as: char word[r].
2. In LINE 1 the array word must be either defined statically or allocated dynamically.
3. LINE 2 can be: mx = malloc(r * sizeof(char **)).
4. LINE 3 can be: mx[i][j] = strdup (word).
5. LINE 2 can be: mx = (char ***)malloc(r * sizeof(char **)).
6. LINE 2 can be: mx = (char **)malloc(r * sizeof(char *)).
7. LINE 3 can be: strcpy (mx[i][j], word).
8. LINE 2 can be: mx = malloc(r * sizeof(char *)).

Response

1. FALSE.
2. TRUE.
3. TRUE.
4. TRUE.
5. TRUE.
6. FALSE.
7. FALSE.

Exercise 10 (2.0 points)

Analyze the following program and indicate the exact output it generates. Please, report the exact program output with no extra symbols.

```

void f (int);
void fp (int);
void fm (int);

```

```

void f (int n) {
    if (n%2==0) {
        fp (n);
    } else {
        fm (n);
    }
    return;
}

void fp (int n) {
    if (n<=0) {
        return;
    }
    printf ("+");
    f (n-1);
    return;
}

void fm (int n) {
    if (n<=0) {
        return;
    }
    printf ("-");
    f (n-1);
    return;
}

int main () {
    f(10);
    return 1;
}

```

Response

+--+--+--+--

Exercise 11 (3.0 points)

A square matrix m of size n stores only capital alphabetic characters. Write the function

```
void check (char **m, int n);
```

which receives the matrix m, its size n, and displays the row including the same alphabetic characters more times. The function displays nothing if no row contain the same character more than once. The function displays a row at choice if more rows contain a character the same number of times.

For example, if the matrix is the following one (with size n=6):

```

X A E I O U
U X A E I O
O U X A E I
I O U X A E
E I O U X A
X Y Z X Y X

```

the last row includes three letters X, thus the function must display that row, i.e., 5.

```
X Y Z X Y X
```

Write the entire program using standard C libraries but implement all required personal libraries. Modularize the program adequately, and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward programs, complex or impossible to understand, will be penalized in terms of the final evaluation.

Solution

To be done.

Exercise 12 (5.0 points)

A file stores an undefined number of rows. Each row includes two fields: An integer value, and a string of at most 100 characters. Those fields are separated by one or more spaces.

Write a C function able to organize the file content into a BST in which each node points to a list. The keys of the BST are integer values and the lists store strings. More.texifically, all record of the file with the same integer value must be stored into the list of the same BST element.

Write the function:

```
bst_t *insert (char *name);
```

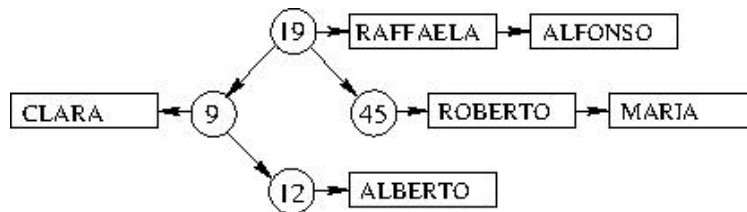
which receives the file name and returns the pointer to such a BST. The lists must not be sorted and may contain repeated elements.

Define the type `bst_t` for the BST nodes, and the type `list_t` for the list nodes. The string within the type `list_t` must be dynamically allocated.

For example, if we suppose the file includes the following rows

```
19 ALFONSO
45 MARIA
9 CLARA
19 RAFFAELA
45 ROBERTO
12 ALBERTO
```

the data structure created by the function must be the following one.



Write the entire program using standard C libraries but implement all required personal libraries. Modularize the program adequately, and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward programs, complex or impossible to understand, will be penalized in terms of the final evaluation.

Solution

To be done.

Exercise 13 (9.0 points)

Write a recursive function

```
void generate (char *name, int n);
```

which stores in the file name all decimal integers of n digits for which:

- the digits of weights 10^0 , 10^2 , 10^4 , 10^6 , etc., have an even value (e.g., 0, 2, 4, 6, 8)
- the digits of weights 10^1 , 10^3 , 10^5 , 10^7 , etc., have an odd value (e.g., 1, 3, 5, 7, 9)
- the sum of all even digits equals the sum of all odd digits.

For example, with $n=4$ digits, the following numbers satisfy specified properties: 1430 (with sum $0 + 4 = 3 + 1 = 4$), 3652 (with sum $2 + 6 = 5 + 3 = 8$), 5676 (with sum $6 + 6 = 7 + 5 = 12$).

The solution must prune the recursion tree as soon as possible, i.e., solutions that generate all integer numbers of n digits and display only the numbers satisfying the requested properties will be penalized in terms of the final mark.

Write the entire program using standard C libraries but implement all required personal libraries. Modularize the program adequately, and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward programs, complex or impossible to understand, will be penalized in terms of the final evaluation.

Solution

To be done.